

Status

Proposed

Context

S2PNexus is a multi-tenant Source-to-Pay platform. ADR-0002 (Universal Supplier Lifecycle & Approval Architecture) and ADR-0003 (Universal Form & Questionnaire Engine) assume a fixed schema for core business objects such as Supplier, Purchase Requisition, Purchase Order, and Contract. In practice, every enterprise customer requires additional fields, custom dropdowns, localized labels, field-level visibility rules, validation rules, and data classification tags specific to its industry, regulatory environment, and internal processes.

Without a platform-level metadata capability, each customization request becomes a one-off development effort requiring schema migration, code changes, and a new deployment. This does not scale across a multi-tenant SaaS customer base and creates upgrade and maintenance risk in the form of customer-specific code forks.

Decision

Implement a Universal Metadata Engine as a core platform service that allows tenant administrators to extend, configure, and govern business object schemas without code changes or database migrations. All core business objects register with the Metadata Engine and inherit custom field support, picklist management, localization, visibility rules, validation rules, and data classification.

Core Capabilities

- Custom Field Definition, scoped per tenant and per object
- Custom Picklists and Dropdowns, tenant-managed value lists
- Field Localization: multi-language labels and help text
- Field-level Visibility and Layout Rules, by role, business unit, or condition
- Validation Rule Engine: required, regex, range, cross-field, custom expressions
- Data Classification and Sensitivity Tags: PII, Confidential, Financial, Restricted
- Schema-less Extension Storage so extensions do not require ALTER TABLE
- Metadata Versioning and Change History
- Metadata-driven UI rendering, integrating with the Form Engine defined in ADR-0003

Supported Business Objects

- Supplier / Supplier Request
- Purchase Requisition / Purchase Order
- Contracts
- Invoices
- Strategic Sourcing Events
- Projects
- Future business objects, via a metadata registration contract

Custom Field Types

Type	Notes
Text / Long Text / Rich Text	Standard entry fields

Type	Notes
Number / Currency / Percentage	Numeric fields with locale-aware formatting
Date / Time / DateTime	Time-zone aware
Single Select / Multi Select / Checkbox	Backed by tenant-managed picklists
Lookup / Reference	References core or custom objects
Calculated / Derived	Computed from other fields via expressions
Attachment	Subject to classification and retention rules

Data Classification and Governance

- PII, Confidential, Financial, and Public classification tags applied at the field level
- Classification drives masking, encryption at rest, export controls, and audit logging; see ADR-0006 Security & Governance
- Field-level access control layered on top of RBAC, for example masking a tax ID for non-privileged roles
- Retention policy tagging at the field and object level

Localization

- Field labels, help text, and picklist values are translatable per locale
- Fallback language chain: tenant default, then platform default
- Adding a language requires no schema change

Extensibility Without Schema Changes

- Custom fields are stored in a schema-less extension store, such as JSONB or an EAV model, keyed to object, tenant, and field definition
- The core relational schema remains stable across tenants and platform upgrades
- Metadata definitions are tenant-scoped and versioned; platform upgrades must not overwrite tenant customizations

Architecture Principles

- Configuration over code: no custom field, dropdown, or validation rule should require a deployment
- Metadata is first-class and API-addressable; other platform services read the same metadata contract
- Backward compatible: adding a custom field must never break existing integrations or reports
- Tenant isolation: metadata definitions never leak across tenants

Non-Functional Requirements

- Custom field creation must not require downtime or a code deployment
- The metadata read path must be cacheable, given that every form and object render depends on it
- The maximum number of custom fields per object per tenant must be configurable, not hardcoded
- All metadata changes must be audited: who changed what, and when

Consequences

Positive: enables true multi-tenant self-service configuration, eliminates the majority of customization requests that would otherwise require custom code, and provides a single source of truth for schema consumed by the Form Engine, Approval Engine, Reporting, and the Integration Platform.

Negative: schema-less extension storage adds query complexity and potential performance overhead relative to native columns, and requires an explicit indexing strategy for reporting at scale. It also introduces a new dependency that most other services must integrate against, increasing initial build complexity.

Alternatives Considered

- Per-tenant schema forks or custom database columns: rejected as unsustainable at scale and disruptive to platform upgrades
- Configuration only through support tickets or professional services: rejected as it does not scale and contradicts the SaaS self-service model
- Fully dynamic schema-per-tenant, using a separate database schema per tenant: rejected as operationally expensive and disruptive to cross-tenant platform upgrades and shared reporting

Dependencies and Related ADRs

- ADR-0002: the Approval Engine should be able to reference custom fields in approval routing conditions
- ADR-0003: the Form Engine renders and validates fields sourced from Metadata Engine field and question definitions; the boundary between the two needs to be explicit
- ADR-0005: external integrations must be able to read and write custom field values through the API without hardcoding field names
- ADR-0006: data classification tags drive encryption, masking, and audit policy

Open Questions

- Where is the boundary between a Form Engine question, as defined in ADR-0003, and a Metadata Engine custom field? Are they the same underlying primitive?
- Does every tenant get self-service metadata changes, or does it require an admin approval workflow for regulated tenants?
- How do custom fields surface in the reporting and BI layer without per-tenant schema drift?